

Search Typeahead System Report

A high-performance, distributed typeahead suggestion system designed to handle high write pressure, serve low-latency suggestions, and provide recency-boosted trending rankings.

1. Architecture Explanation

The system is built on a split architecture consisting of a persistent primary SQL database, a logical distributed caching layer managed by a consistent hashing ring, a thread-safe write buffering manager, and a real-time analytics web dashboard.

Distributed Cache & Consistent Hashing Ring

To simulate a production-grade distributed environment, we instantiate 3 logical caching nodes: Cache-Node-A, Cache-Node-B, and Cache-Node-C. To map incoming search prefixes evenly across these nodes, we implement a Consistent Hashing Ring with 50 virtual nodes per cache node (150 total). A search prefix key (e.g., 'py') is hashed using MD5 into a 128-bit space. The query is routed to the first virtual cache node located clockwise from the prefix hash value. Virtual nodes ensure that cache keys are balanced uniformly across physical servers, mitigating hot-spotting.

Trending & Recency-Aware Ranking

The typeahead system supports two ranking mechanisms: Basic and Enhanced (Trending). Basic mode sorts queries starting with a prefix purely by historical search count. Enhanced mode calculates a score combining historical volume and recent searches with a linear decay time window:

$$\text{Score}(q) = \text{HistoricalCount}(q) + \sum_{s \in \text{RecentSearches}(q)} \text{Max}(0, 1 - (t_{\text{now}} - t_s) / \text{window_size}) * \text{boost_factor}$$

This decay scoring ensures queries searched recently climb to the top of suggestions immediately and drop back to their historical baseline score after the time window (e.g., 60 seconds) expires. It prevents temporary spikes from permanently dominating search rankings.

Batch Write Buffer

To protect the database from lock contention and high disk write overhead, we introduce a thread-safe BatchWriteManager. Search submissions are intercepted and placed into an in-memory queue. The queue is flushed to the database only when the queue size reaches a threshold (default: 10) or a period interval is met (default: 5 seconds). During a flush, duplicates are aggregated, and a single SQLite transaction upserts historical counts and inserts recent timestamp logs, minimizing disk write transactions.

2. Dataset Source & Loading Instructions

The system seeds an initial SQLite database ('search_database.db') with a rich dataset of 5,200 unique queries across programming languages, e-commerce products, recipes, movies, travel destinations, and general search queries. Frequencies are generated using a Zipf-like popularity distribution.

Loading Instructions:

- **Generate Database:** Run 'python generate_dataset.py' to rebuild the SQLite database and seed the tables.
- **Verify Database:** Upon execution, a database file ('search_database.db') is populated containing 'queries' (query, historical_count) and 'recent_searches' (query, timestamp) tables.

3. API Documentation

- **GET /suggest?q=<prefix>:** Fetches the top 10 typeahead suggestions matching the prefix. Basic/Enhanced scoring modes apply.
- **POST /search:** Registers a search query, adding it to the in-memory write buffer. Returns 'Searched'.
- **GET /cache/debug?prefix=<prefix>:** Returns cache debug stats: responsible cache node, hit/miss status, TTL time remaining, and MD5 hash fraction.
- **GET /cache/ring:** Exposes the exact hash ring fractions and node mappings for all 150 virtual nodes.
- **GET /metrics:** Returns live stats: avg/p95 suggestion latency, cache hits/misses, DB reads, DB writes saved, and current buffer list.
- **GET /trending:** Fetches the top 10 popular queries and top 10 recency-boosted trending queries.

4. Explanations of Design Choices & Trade-offs

Buffer-Based Batching vs Synchronous Writes

- **Buffered Batching:** Saves disk I/O operations by grouping queries into bulk transactions. Improves suggestion performance by freeing the database from write locks.
- **Trade-off (Data Loss):** If the server crashes, unflushed searches in memory are lost. We accept this trade-off because search frequency tracking is non-critical (unlike financial transactions). WAL or a message queue (Kafka/Redis) can mitigate this in production.

Consistent Hashing vs Simple Hashing

- **Consistent Hashing:** When adding/removing cache nodes, only K/N keys are remapped. Standard hashing ($\text{hash}(k) \% N$) invalidates the entire cache, overloading the DB.
- **Virtual Nodes:** Adding 50 virtual nodes per server balances the keys uniformly across all cache memory banks, preventing hot-spotting.

5. Performance & Automated Verification Report

A verification script ('verify_system.py') was executed to validate the core system modules. The test suite outputs show that all performance expectations were successfully met:

```
=====
STARTING SEARCH TYPEAHEAD SYSTEM VERIFICATION SUITE
=====

[TEST 1] Verifying Consistent Hashing Ring...
[+] Prefix 'py' consistently routes to 'Node-C'
[+] Prefix 'con' consistently routes to 'Node-A'
[+] Prefix 'dist' consistently routes to 'Node-C'
[+] Prefix 'web' consistently routes to 'Node-C'
[+] Prefix 'mach' consistently routes to 'Node-B'
[+] Ring distribution: {'Node-A': 50, 'Node-B': 50, 'Node-C': 50}
[+] Hashing ring verified successfully.

[TEST 2] Verifying Cache Node behavior...
[+] Cache miss handled correctly.
[+] Cache hit value retrieved correctly.
[*] Waiting for TTL expiration (1.7s)...
[+] Cache TTL expiration verified successfully.

[TEST 3] Verifying Batch Write Buffer...
[*] Submitting 3 searches (below threshold of 5)...
[+] Confirmed database has NOT been written to yet (write reduction in progress).
[*] Submitting 2 more searches (triggering threshold 5)...
[+] Confirmed buffer flushed. DB count is now: 5
[*] Submitting 2 more searches and waiting 2.5s for time flush...
[+] Confirmed periodic flush. DB count is now: 7
```

```
[+] Batch write logic verified successfully.
```

```
[TEST 4] Verifying Trending Searches Recency-Aware Scoring...
```

```
[+] Basic mode sorting verified: trending_test_popular ranks first (Score: 100)
```

```
[*] Enhanced Suggestions: [('trending_test_recent', 509), ('trending_test_popular', 100)]
```

```
[+] Enhanced mode sorting verified: trending_test_recent boosted to first (Score: 509)
```

```
[*] Waiting for recency window decay (11s)...
```

```
[*] Suggestions after decay: [('trending_test_popular', 100), ('trending_test_recent', 10)]
```

```
[+] Decay verified successfully: trending_test_popular returned to rank 1.
```

```
=====
```

```
ALL TESTS PASSED SUCCESSFULLY! SYSTEM LOGIC CORRECT.
```

```
=====
```